LOC-AI: The Comprehensive Developer Manual

Version: 1.0.0 (Master Build) **Developer:** Lochana Chamod **Core Engine:** Electron + Node.js **AI Backends:** Ollama (Offline), Groq Llama-3.3 (Online)

---

1. Application Overview

LOC-AI is a dual-engine coding assistant designed for professional developers. It solves the critical problem of relying solely on cloud-based AI by integrating a robust offline engine.

- **God Mode (Online):** Connects to the Groq API (Llama-3.3-70B), offering the world's fastest inference speeds for complex architectural problems and modern framework knowledge.
- **Bunker Mode (Offline):** Utilizes a local Ollama instance running qwen2.5-coder, ensuring total privacy and functionality without an internet connection.
- **Cyber-Glass Interface:** A custom-built UI featuring neon aesthetics, glassmorphism, and advanced typography designed for readability and visual appeal.

---

2. Project Structure & File Manifest

This section details every file in your source code and its specific responsibility.

package.json

- **Role:** The Project Manifest.
- **Function:** Tells Node.js what libraries to install (electron, electron-builder, highlight.js, marked). It defines the start command (for testing) and the build command (for creating the EXE). It also sets the app metadata (Author: Lochana Chamod).

main.js

- **Role:** The Skeleton / Main Process.
- **Function:** This is the entry point of the Electron app. It creates the actual browser window, removes the standard Windows frame (to allow our custom design), sets

the app icon, and handles system-level events like minimizing or closing the window.

index.html

- **Role:** The Skeleton / Structure.
- **Function:** Defines the layout of the app. It holds the Sidebar, Chat Area, Input Zone, and the hidden Settings Modal. Crucially, it contains the **Content Security Policy (CSP)** meta tag that whitelists connections to 127.0.0.1 (Ollama) and api.groq.com.

styles.css

- **Role:** The Skin / Design.
- **Function:** Contains all the visual rules. It defines the "Cyber-Glass" variable colors, the neon glow effects, the custom scrollbars, and the "Orbitron" font usage. It also includes the specific typography fixes that ensure AI responses have proper spacing between paragraphs and headers.

renderer.js

- **Role:** The Brain / Logic.
- **Function:** This is the most complex file. It handles:
    - **AI Routing:** Decides whether to send your prompt to queryLocalAI or queryGroqAI.
    - **Context Management:** Reads the last 6 messages to give the AI "memory."
    - **File Handling:** Uses fs and webUtils to securely read text files dropped onto the window.
    - **Markdown Parsing:** Converts raw text into beautiful HTML with colored code blocks.
    - **UI Logic:** Handles button clicks, mode switching, and the thinking indicator.

storage.js

- **Role:** The Memory / Database.
- **Function:** Creates a hidden folder (.loc-ai) in your user directory. It saves your chat history to a JSON file so your conversations persist even after you restart the computer. It also handles deleting chats.

---

3. Operational Guide

Switching Modes

- **Bunker Mode:** Activated by default or by clicking the shield icon. Ensure Ollama is running in the background (check system tray).
- **God Mode:** Click the lightning bolt icon. If it's your first time, the settings menu will appear asking for your Groq API key.

File Analysis

- Drag and drop any code file (.py, .js, .txt, .cpp) directly onto the chat window. A blue chip will appear.
- Type your question (e.g., "Find the bug"). The app transparently injects the file content into the AI's prompt.

Settings & Customization

- Click the "Settings" button in the sidebar to change your API key or update the **System Persona** (e.g., instructing the AI to "Answer only in JSON").

---

4. Updating the EXE (Maintenance)

If you want to change the code (e.g., update the logo, change colors, or switch to a newer AI model), follow this exact process to create a new installer.

Step 1: Edit the Code

- Open the project in VS Code.
- Make your changes (e.g., edit styles.css to change neon blue to neon red).

Step 2: Clean the Build Folder

- Delete the **dist** folder in your project directory. This ensures no old files interfere with the new build.

Step 3: Re-Build

- Open the Terminal in VS Code.
- Run:

Bash

npm run build

Step 4: Distribute

- The new LOC-AI Setup 1.0.0.exe will be in the fresh dist folder. You can share this file with anyone; they do not need Node.js or VS Code to run it.

---

5. Troubleshooting Common Issues

- **Error: "Failed to Fetch" (Offline Mode)**
    - **Cause:** Ollama is not running.
    - **Fix:** Open Start Menu -> Type "Ollama" -> Run it. Check the system tray.
- **Error: "Failed to Fetch" (Online Mode)**
    - **Cause:** Internet issue or Invalid API Key.
    - **Fix:** Check your internet. If that works, go to Settings -> Clear the API key -> Enter a new valid Groq key.
- **Error: Build Fails with "Permission Denied"**
    - **Cause:** Windows is blocking the file creation.
    - **Fix:** Close VS Code. Right-click VS Code -> "Run as Administrator". Run npm run build again.